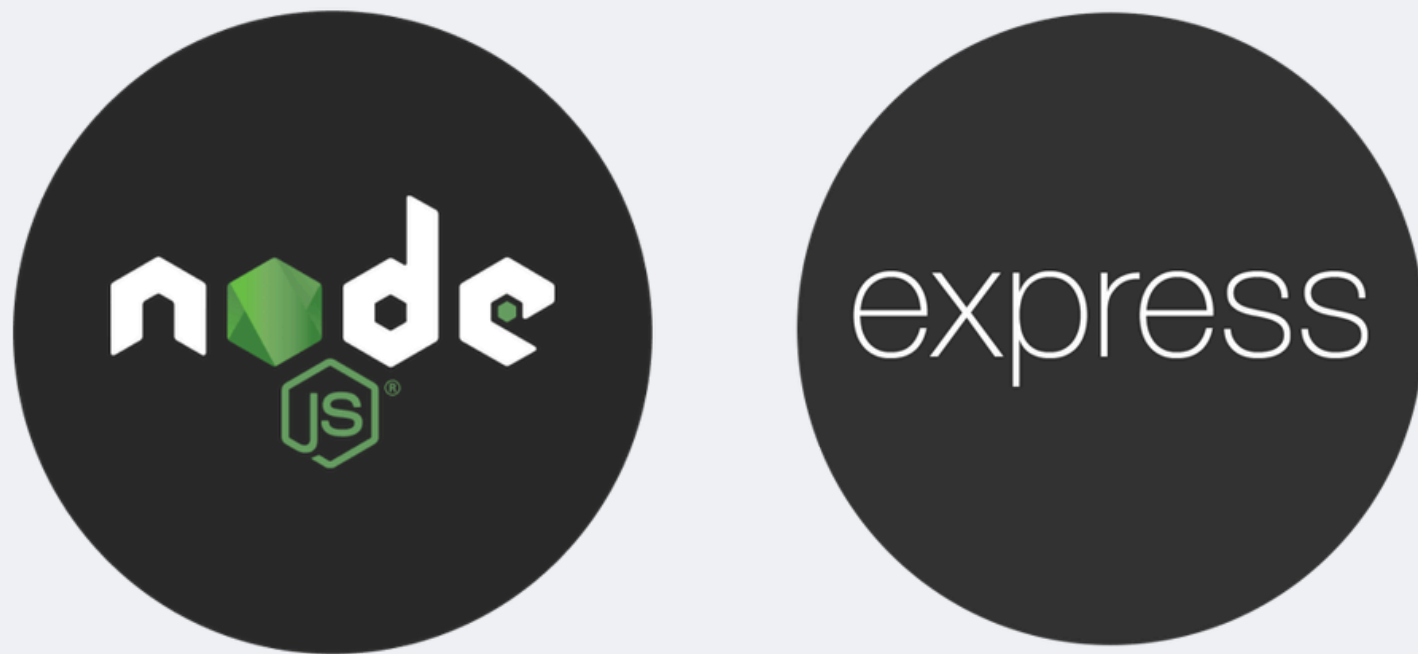




SE 321

Lecture 7: Intro to **Node.js** + **Express**

Sadia Sultana

Lecturer
Department of Software Engineering
Daffodil International University



Introduction to Node.js & Express

Building the backend that powers every app

Server

Routing

Controllers

Middleware

REST

Learning roadmap

07

1

Node & Express

What runs your
JavaScript on a server

2

Project setup

package.json, npm,
server.js

3

Routing

Map each URL to a
response

4

Controllers

Where the real logic
lives

5

Middleware & tools

CORS, dotenv,
nodemon

Core idea

Express turns each incoming HTTP request into a simple function you write. You decide what every URL returns — that is your backend.

The Problem: The Browser Can't Do It Alone

React draws the screen — but where does the data actually live?



Data must persist

A refresh wipes React state. Users, posts and orders have to live in a database on a server that never forgets.



Secrets can't ship to the browser

API keys, passwords and payment logic must stay on the server, where visitors can't open dev-tools and read them.



One shared source of truth

Your website, phone app and other clients all need the same data from one central place they can call.



Think of posting a photo

Your browser sends the image to a **server**. The server stores it in a database, then hands it back to every follower's feed — **the browser alone could never do this.**

The backend's job: store data, guard secrets, and answer requests — so the frontend can just ask and display.

So, What Exactly Is Node.js?

JavaScript that runs on a server, not just inside the browser



Node.js lets you run JavaScript *outside* the browser. It is built on Chrome's V8 engine, so the same language you use for the frontend now powers your server too.



One language

Write the client and the server in JavaScript — no context-switch between two languages.



Non-blocking

Handles thousands of connections at once without waiting on any single one.



Huge ecosystem

npm has 2M+ ready-made packages — most problems are one install away.

REAL WORLD: Netflix, LinkedIn, PayPal, Uber and NASA all run Node.js on their servers.

And What Is Express.js?

A minimal framework that makes Node web servers simple



Express is the most popular web framework for Node. Raw Node's http module is verbose; Express gives you clean routing and middleware in just a few lines.



Routing

Map each URL and method to a handler function that runs on a match.



Middleware

Run shared code on every request — logging, auth, JSON parsing.



Unopinionated

Add only what you need. You design the folders and the structure.

REAL WORLD: Express is the 'E' in the classic MERN and MEAN stacks used across the industry.

Project Structure, package.json & npm

How a backend project is organised on disk

my-api/ (folder tree)

```
my-api/
├─ node_modules/    # installed packages
├─ src/
│   ├─ routes/
│   ├─ controllers/
│   └─ app.js
├─ server.js        # entry point
├─ .env             # secrets
└─ package.json
```

package.json

```
{
  "name": "my-api",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": { "express": "^4" }
}
```

npm — Node Package Manager

npm init -y creates package.json.

npm install express downloads a package into node_modules and records it as a dependency.

Your First Server

Just a few lines to a running web server

server.js

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Hello from Express!');
});

app.listen(3000, () => {
  console.log('Server on port 3000');
});
```

require('express')

Loads the Express package you installed with npm.

express()

Creates your app — the object you attach every route to.

app.listen(3000)

Starts the server and waits for requests on a port.

Run it

node server.js — then open <http://localhost:3000>

Routing: Responding to URLs

Mapping a URL + method to a handler function

● ● ● routes.js

```
// app.METHOD(path, handler)
app.get('/products', (req, res) => {
  res.send('List of products');
});

app.post('/products', (req, res) => {
  res.send('Product created');
});

app.get('/about', (req, res) => res.send('About'));
```

Method = intent

GET reads, POST creates, PUT updates, DELETE removes.

Path = the URL

'/products' matches yoursite.com/products exactly.

Handler = (req, res)

The function Express runs whenever that route matches.

REAL WORLD: Every button and link on a site quietly hits a route just like these.

The Request & Response Objects

Two objects every handler receives: req and res

● ● ● handler.js

```
app.get('/search', (req, res) => {  
  // read what the client sent  
  const term = req.query.q;  
  // url: ..?q=phone → term = phone  
  const auth = req.headers;  
  
  // send a reply back  
  res.status(200).json({  
    message: 'Results for ' + term  
  });  
});
```

req = the question (client's request)

Everything the client sent: params, query, body, headers, url, etc.

res = your answer (server's response)

How you reply: res.send(), res.json(), res.status().

req.query

Values that come after ? in the URL (?q=phone).

res.json()

Sends data as JSON — the native language of APIs.

The Request & Response Objects

The req (Request) object represents the incoming HTTP request from a client. You can use it to access critical data:

● ● ● handler.js

```
app.get('/search', (req, res) => {  
  // read what the client sent  
  const term = req.query.page;  
  // url: /search?page=2 -> term = 2  
  
  const auth = req.headers;  
  
  // send a reply back  
  res.status(200).json({  
    message: 'Results for ' + term  
  });  
});
```

req.body

Holds data submitted in the request body (e.g., JSON payloads or form submissions)

req.params

Extracts dynamic routing parameters from the URL path (e.g., getting 123 from /user/:id via req.params.id)

req.query

Contains key-value pairs of the URL query string parameters (e.g., getting 2 from /search?page=2 via req.query.page)

**req.headers, req.method, req.cookies,
req.url, req.protocol and many more...**

Route Parameters

Dynamic URLs using :placeholders

params.js

```
// :id is a placeholder slot
app.get('/users/:id', (req, res) => {
  const id = req.params.id;
  res.send('User ' + id);
});

// /users/42 -> req.params.id = '42'

app.get('/posts/:year/:month', (req, res) => {
  res.json(req.params);
});
```

:id

A named slot in the path that matches any value.

req.params

An object holding every route parameter by name.

Chain many

Combine several: /posts/:year/:month.

REAL WORLD: youtube.com/watch/:videoid — one route serves every single video.

Controller Functions

Move the logic out of your route file

● ● ● split into two files

```
app.get('/users/:id', (req, res) => {  
  const id = req.params.id;  
  res.send('User ' + id);  
});
```

// Usually, we don't attach the controller function directly with the route. We separate it into another folder (controllers), following the rules of MVC.

```
// controllers/user.js  
const getUser = (req, res) => {  
  res.json({ id: req.params.id });  
};  
module.exports = { getUser };
```

Controller

A named function that holds the real work of a route.

module.exports

Shares the function so other files can import it.

Why bother?

Cleaner, testable, reusable code as the app grows.

Controller Functions

Move the logic out of your route file

● ● ● split into two files

```
// controllers/user.js
const getUser = (req, res) => {
  res.json({ id: req.params.id });
};
module.exports = { getUser };

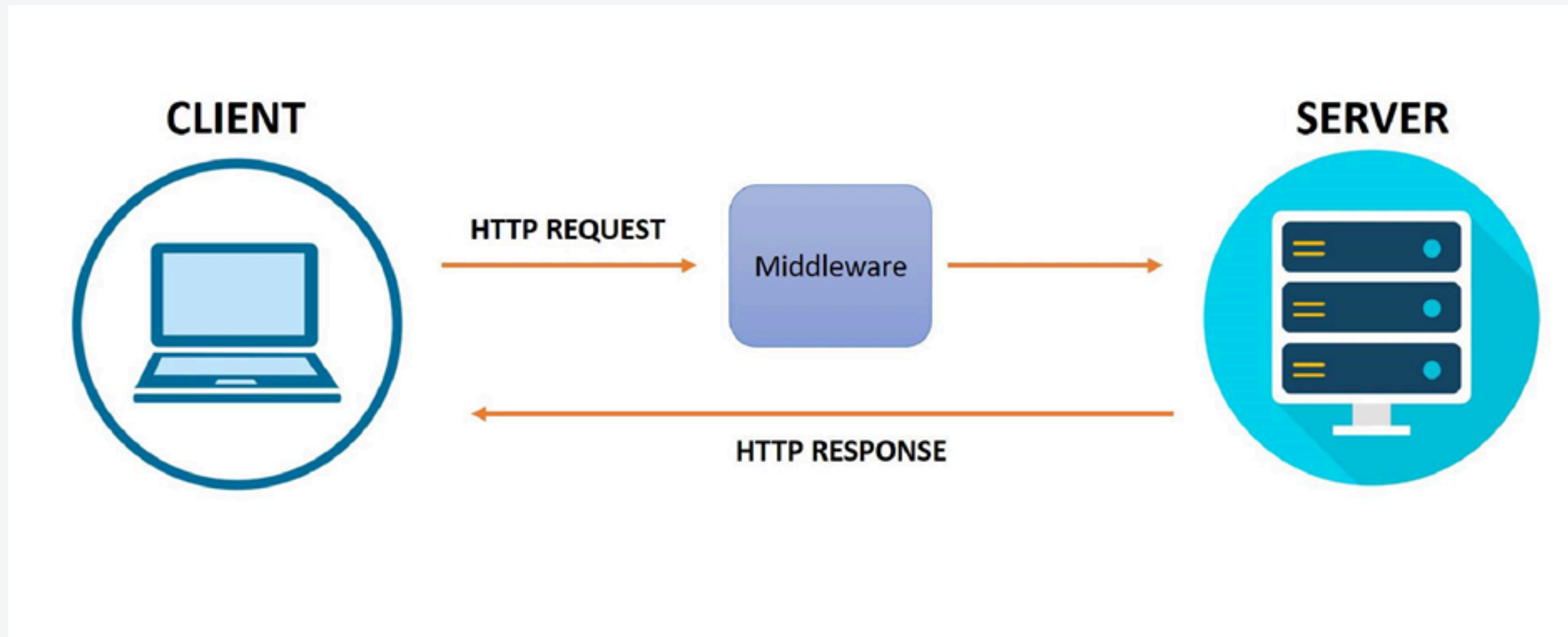
// routes/user.js
const { getUser } = require('../controllers/user');
router.get('/users/:id', getUser);
```

Thin routes

The route just points a URL to its controller — nothing more.

Middleware: The Assembly Line

Functions that run between the client's request and the server's response



Middleware: The Assembly Line

Functions that run between the client's request and the server's response



The (req, res, next) trio

Middleware gets a third argument, next. Calling next() passes control to the next function in the chain.



Runs on the way in

Logging, authentication and JSON body-parsing all happen before your route handler ever runs.



```
middleware.js
```

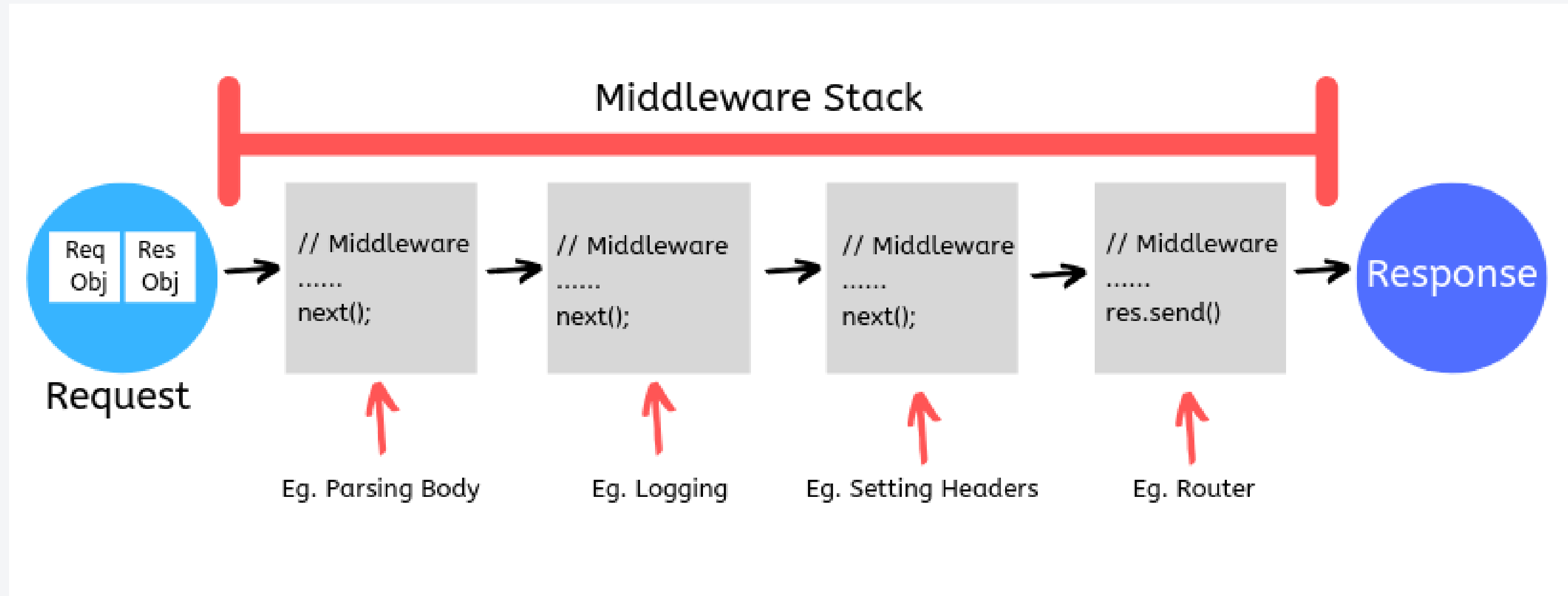
```
// a custom middleware
function logger(req, res, next) {
  console.log(req.method, req.url);
  next(); // hand off to next step
}
```

```
app.use(logger);
app.use(express.json());
```

app.use()

Registers a middleware so it runs on every incoming request.

Middleware Stack



The Essential Toolkit

Three packages nearly every Express project relies on



CORS

Lets your React app on a different origin call this API without the browser blocking it.

```
app.use(cors());
```



dotenv

Keeps secrets out of your code in a .env file, read via process.env.

```
require('dotenv').config();
```



nodemon

Auto-restarts the server every time you save — no manual stop and start.

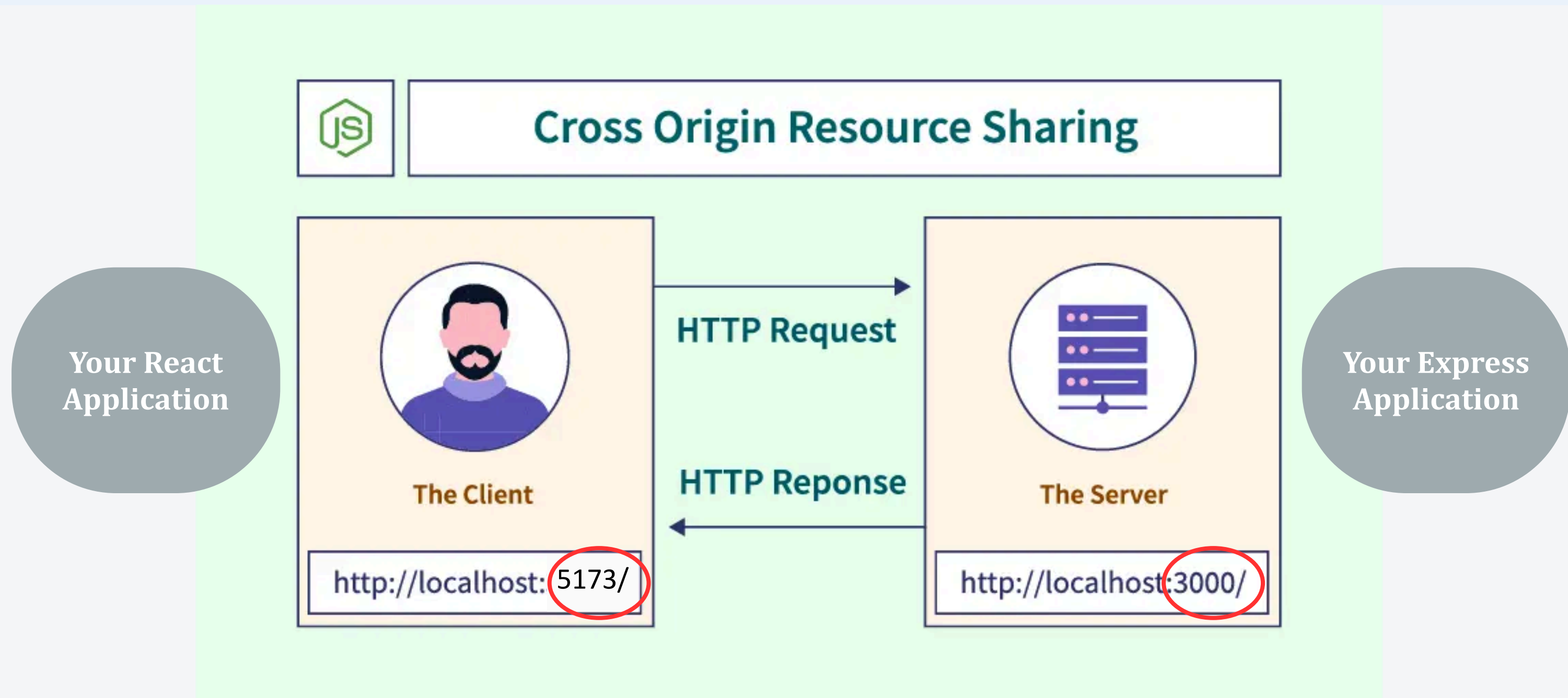
```
nodemon server.js
```

Install them: `npm install cors dotenv` `npm install --save-dev nodemon`

What does CORS do?



CORS (Cross-Origin Resource Sharing) is a browser security mechanism. By default, browsers enforce a "**same-origin policy**", meaning a web page cannot request data from a different domain. CORS allows servers to explicitly permit trusted domains (clients) to access their resources using specific **HTTP headers**.



What does CORS do?



CORS (Cross-Origin Resource Sharing) is a browser security mechanism. By default, browsers enforce a "**same-origin policy**", meaning a web page cannot request data from a different domain. CORS allows servers to explicitly permit trusted domains (clients) to access their resources using specific **HTTP headers**.

● ● ● CORS

```
// import CORS
const cors = require('cors');
// set the client origin
const corsOptions = {
  origin: ["http://localhost:5173/"]
}
// use the middleware
app.use(cors(corsOptions));
```

origin: ["http://localhost:5173/"]

The server includes this origin in the HTTP header, which allows the client browser to read/fetch information from the server.

Everything Together: A Tiny Notes API

```
server.js

const express = require('express'); const cors = require('cors');
const app = express();

app.use(cors());           // allow cross-origin
app.use(express.json());   // parse JSON body

let notes = [];           // pretend database

app.get('/notes', (req, res) => res.json(notes));

app.post('/notes', (req, res) => {
  notes.push(req.body.text);
  res.status(201).json({ added: req.body.text });
});

app.get('/notes/:id', (req, res) => res.json(notes[req.params.id]));

app.listen(3000, () => console.log('Server running on PORT 3000'));
```

Spot every concept:

- ✓ **Server setup** `express()` + `listen`
- ✓ **Middleware** `cors`, `express.json`
- ✓ **Routing** `get` / `post`
- ✓ **Route params** `:id`
- ✓ **Req / Res** `req.body`, `res.json`



You Can Now Build a Backend



Node & Express — run JS on a server, framework on top



Routing — app.get / post map URLs to handlers



Req & Res — read req.params/query/body, reply res.json



Controllers — keep the logic out of route files



Middleware — (req, res, next) runs on every request



Toolkit — CORS · dotenv · nodemon

Thank You!